

ELEVANCE SKILLS

Elite Embedded Systems Internship Challenge

CANDIDATE EVALUATION REPORT

Smart Battery Management System | ESP32 + IoT

Candidate	Vaibhavi BH
Email	vaibhavi.bh21@gmail.com
Challenge	Elite Embedded Systems Internship — BMS Track
Date	June 2025
Evaluator	Elevance Skills Technical Assessment Panel

FINAL VERDICT	PASS	STIPEND	NOT RECOM MENDED	OVERALL SCORE	8.0 / 10
------------------	------	---------	---------------------	------------------	----------

- Health classification shows an attempt at engineering-grade thinking.
- Code organization reflects modular intent.

Areas for Improvement

- `delay()` usage introduces blocking behavior — unacceptable in production BMS firmware; must be replaced with non-blocking timer patterns (`millis()` or FreeRTOS tasks).
- Tolerance and threshold handling is incomplete — missing dynamic tolerance bands essential for accurate SoH estimation.
- Pack analysis lacks edge-case handling for cell imbalance or partial failure scenarios.

Compliance Assessment

Technically reviewed as an isolated module. Integration with other tasks could not be verified.

Task 2 — Event-Driven Safety Protection Kernel

Score: 7.5/10

Objective

Build a safety kernel with real-time event detection, fault response, and recovery logic for over-voltage, under-voltage, and thermal events.

What Was Implemented

- Safety monitoring with threshold-based detection.
- Weak-cell and overvoltage detection logic.
- Basic recovery flow.

Strengths

- Core safety monitoring concept is correctly understood and structurally implemented.
- Overvoltage and weak-cell detection covers the primary fault categories expected.

Areas for Improvement

- Recovery logic is simplified and does not reflect automotive-grade resilience expectations (e.g., ISO 26262 fault containment principles are not evident).
- The kernel is not truly event-driven — fault detection appears to follow a polling model rather than interrupt/callback-driven architecture.
- No evidence of hierarchical fault prioritization or multi-fault concurrency handling.

Compliance Assessment

Reviewed in isolation. Safety kernel integration with state management layer (Task 4) could not be assessed.

Task 3 — Intelligent Embedded HMI & Diagnostic Interface

Score: 7.5/10

Objective

Develop an embedded Human-Machine Interface using an LCD with multi-screen navigation and real-time diagnostic display.

What Was Implemented

- LCD interface with multi-screen design.
- Multiple screen modes for displaying different data views.
- Basic refresh implementation.

Strengths

- Multi-screen concept is correctly approached and demonstrates HMI design awareness.
- Screen separation by data category is a good UX decision for embedded displays.

Areas for Improvement

- Refresh optimization is limited — risk of LCD flicker under real-time update loads is not mitigated.
- No evidence of intelligent display prioritization (e.g., elevated fault data taking precedence over routine telemetry).
- Buffered rendering or dirty-flag update patterns should be applied to reduce unnecessary screen writes.

Compliance Assessment

Reviewed as standalone HMI module. Real-time data binding to live system state could not be verified.

Task 4 — Fault-Tolerant Embedded Runtime System

Score: 8.0/10

Objective

Implement a fault-tolerant runtime with defined operating modes, a fault manager, and operational logging.

What Was Implemented

- Multiple runtime operating modes defined.
- Fault manager with classification logic.
- Logging implementation for events and faults.

Strengths

- Runtime mode concept is correctly structured — normal, warning, and fault modes are addressed.
- Fault manager demonstrates systematic thinking about failure classification.
- Logging adds traceability, which is a production-relevant engineering practice.

Areas for Improvement

- Some fault verification logic appears simulated rather than hardware-triggered — reduces real-world validity.
- State machine implementation lacks formal guard conditions and transition validation.
- Logging format should be structured (timestamped, severity-tagged) for production readiness.

Compliance Assessment

Runtime and fault modes reviewed in isolation. Behavioral coupling to other modules (safety kernel, telemetry) could not be verified.

Task 5 — Intelligent Cloud Telemetry Architecture

Score: 8.0/10

Objective

Build a cloud connectivity layer with Blynk integration, WiFi resilience, and intelligent event-driven data transmission.

What Was Implemented

- Blynk IoT platform integration.
- WiFi connectivity monitoring.
- Reconnection logic on WiFi loss.
- Partial event-driven telemetry.

Strengths

- Blynk integration is functional and demonstrates basic cloud connectivity understanding.
- WiFi monitoring and reconnect logic addresses a real-world reliability concern.

Areas for Improvement

- Event-driven telemetry is only partially realized — telemetry transmission does not appear fully decoupled from polling cycles.
- No evidence of data queuing or offline buffering for periods of connectivity loss.

5. TECHNICAL STRENGTHS

- Full coverage of all six challenge modules — demonstrates persistence, time management, and a systematic approach to engineering tasks.
- Consistent modular code structure across tasks — avoids monolithic design and reflects awareness of software engineering principles.
- Use of multiple source files and separation of concerns — shows an understanding of firmware architecture beyond beginner-level thinking.
- Documentation quality — candidate invested effort in making the work understandable, which is a professional engineering habit.
- Fault management concepts implemented across tasks 2 and 4 — safety-awareness is present and structurally integrated.
- Cloud telemetry and dashboard delivery — candidate successfully engaged with both embedded and IoT layers of the challenge, demonstrating cross-domain competence.
- LCD HMI design with multi-screen navigation — shows practical embedded UI thinking beyond simple serial output.

6. AREAS FOR GROWTH

System Integration

The most critical skill gap. Building individual modules is fundamentally different from integrating them into a single, coherent runtime. Future work should prioritize building unified systems where modules share state, communicate via defined interfaces, and behave as one architecture.

Event-Driven Design

Heavy reliance on polling and blocking constructs (`delay()`, synchronous checks) limits the responsiveness and efficiency of embedded systems. Transitioning to interrupt-driven, callback-based, or FreeRTOS task-based patterns is essential for production-grade BMS firmware.

Fault Handling Depth

Fault management should be hierarchical — faults must be prioritized, isolated, and escalated systematically. Simulated fault verification should be replaced by hardware-triggered testing scenarios.

Production-Grade State Machines

State machines require formal definition: states, transitions, guard conditions, and entry/exit actions. Implementing and validating these rigorously is a non-negotiable skill for safety-critical embedded work.

Scalability Thinking

As cell counts and feature sets grow, hardcoded logic, fixed arrays, and non-parameterized thresholds become liabilities. Designing for configurability from the start is a senior engineering discipline worth developing early.

Cloud Architecture Quality

Telemetry should be decoupled from main execution, support offline buffering, and apply bandwidth-efficient transmission strategies. These patterns separate functional IoT from production IoT.

7. SCORING SUMMARY

Task	Module	Score
Task 1	Adaptive Multi-Cell Battery Intelligence Engine	8.0 / 10

Task 2	Event-Driven Safety Protection Kernel	7.5 / 10
Task 3	Intelligent Embedded HMI & Diagnostic Interface	7.5 / 10
Task 4	Fault-Tolerant Embedded Runtime System	8.0 / 10
Task 5	Intelligent Cloud Telemetry Architecture	8.0 / 10
Task 6	Executive Battery Intelligence Dashboard	8.0 / 10
Overall Technical Score		8.0 / 10

8. FINAL VERDICT

VERDICT

PASS

STIPEND ELIGIBILITY

NOT RECOMMENDED

Rationale

- PASS is awarded because the candidate completed all six modules, demonstrated meaningful technical effort, and produced work of reasonable quality within individual tasks.
- Stipend is NOT RECOMMENDED because the submission did not comply with the core architectural requirement: a single integrated BMS project. System-level integration — a primary evaluation objective — could not be assessed.
- Several advanced criteria (automotive-grade safety, true event-driven architecture, production-grade state machines, historical analytics) were only partially satisfied.
- The work demonstrates good foundational potential but does not consistently meet the bar set for stipend selection in this competitive evaluation cohort.

9. CLOSING REMARKS

Vaibhavi, completing all six modules of this challenge is a meaningful accomplishment in itself. The breadth of work submitted — spanning embedded firmware, safety logic, HMI design, fault management, IoT connectivity, and an analytics dashboard — reflects genuine commitment and a willingness to engage with complex engineering systems.

Your modular code structure and documentation quality stand out as strengths that many candidates at this stage have not yet developed. These are professional habits that will serve you well as you progress.

The primary growth area identified — system integration — is not a reflection of limited capability, but rather the next level of engineering maturity to reach for. Building components is stage one. Making them work cohesively as a single system, under shared state and real-time constraints, is stage two. That transition is what separates good embedded engineers from great ones.

We encourage you to revisit this challenge with a focus on unified integration, and to continue building in the embedded systems and IoT space. The foundation you have established here is solid ground to build on.

